

[CI2] Concepts informatiques — Cours 2

Jérémy Ledent

Université Paris Cité, IRIF



INSTITUT
DE RECHERCHE
EN INFORMATIQUE
FONDAMENTALE



Université
Paris Cité

Inscription aux groupes de TD

[Cours](#) [Paramètres](#) [Participants](#) [Notes](#) [Rapports](#) [Plus ▾](#)

 **IFA2E025 Concepts Informatiques 2**

 **Présentation** Tout replier

Ceci est la page Moodle pour le cours [CI2] Concepts Informatiques.

 [Annonces](#)

 [Inscription à un groupe de TD](#)

 **Séances et intervenants**

← **Inscrivez-vous !**

Rappels (expressions arithmétiques)

Une **expression arithmétique** peut être représentée de 4 façons :

- ▶ Comme un arbre syntaxique
- ▶ Notation infixe : $(3 + 2) \times (7 - (2 + 2))$
- ▶ Notation préfixe : $\times + 3 2 - 7 + 2 2$
- ▶ Notation postfixe : $3 2 + 7 2 2 + - \times$

On a vu :

- ▶ Comment convertir la forme infixe vers un arbre.
- ▶ Comment convertir un arbre vers la forme (pré/post)fixe.
- ▶ Comment évaluer une expression sous forme postfixe.

Conversions directes entre les formes infixe et postfixe

(Rappel) Algorithme d'évaluation d'une forme postfixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Conversion postfixe $\rightarrow$ infixe

Algorithme de conversion postfixe $\rightarrow$ infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : $"(" + e2 + \text{opérateur} + e1 + ")"$
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



Conversion postfixe $\rightarrow$ infixe

Algorithme de conversion postfixe $\rightarrow$ infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : `3 2 + 7 2 2 + - ×`



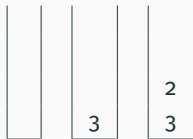
Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : `3 2 + 7 2 2 + - ×`



Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



Conversion postfixe $\rightarrow$ infixe

Algorithme de conversion postfixe $\rightarrow$ infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



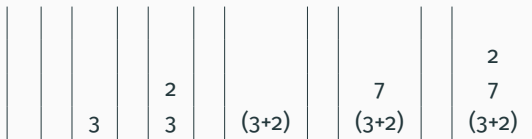
Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : `3 2 + 7 2 + - ×`



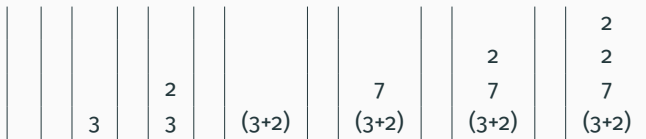
Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : `3 2 + 7 2 + - ×`



Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : `3 2 + 7 2 2 + - ×`

						2	
		2		7	2	2	(2+2)
	3	3	(3+2)	(3+2)	(3+2)	(3+2)	(3+2)

Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$

						2		
		2		7	2	2	(2+2)	
	3	3	(3+2)	(3+2)	(3+2)	(3+2)	7	(7-(2+2))
							(3+2)	(3+2)

Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : `"(" + e2 + opérateur + e1 + ")"`
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$

						2			
		2		7	2	2	(2+2)		
3	3	(3+2)	(3+2)	(3+2)	(3+2)	(3+2)	(3+2)	(7-(2+2))	((3+2)×(7-(2+2)))

Conversion postfixe → infixe

Algorithme de conversion postfixe → infixe

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments $e1$ et $e2$
 - On empile la forme infixe : **"(" + $e2$ + opérateur + $e1$ + ")"**
- ▶ À la fin, la pile contient la forme infixe

Exemple : Forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$

						2			
		2		7	2	(2+2)			
3	3	(3+2)	(3+2)	(3+2)	(3+2)	7	(7-(2+2))		
						(3+2)	(3+2)	((3+2)×(7-(2+2)))	

```
public static String convertToInfix(String[] postfixExpr) {  
    Stack<String> p = new Stack<String>();  
    for (String s : postfixExpr) {  
        if (s.equals("+") || s.equals("-")  
            || s.equals("*") || s.equals("/")) {  
            String e1 = p.pop();  
            String e2 = p.pop();  
            p.push("(" + e2 + s + e1 + ")");  
        } else {  
            p.push(s);  
        }  
    }  
    return p.pop();  
}
```

Conversion infixe $\longrightarrow$ postfixe

Simplification : supposons que l'expression contient des parenthèses partout

✓ $((2 + 3) \times 4)$

✓ $(2 + (3 \times 4))$

✗ $2 + 3 \times 4$

donc pas de priorité des opérations à gérer.

Conversion infixe $\longrightarrow$ postfixe

Simplification : supposons que l'expression contient des parenthèses partout

✓ $((2 + 3) \times 4)$

✓ $(2 + (3 \times 4))$

✗ $2 + 3 \times 4$

donc pas de priorité des opérations à gérer.

Algorithme de conversion infixe $\longrightarrow$ postfixe

On initialise `String result = ""`,

Puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result =`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result =`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result =`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2`



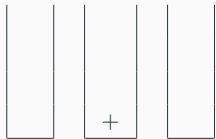
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 +`



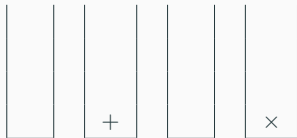
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérateur, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 +`



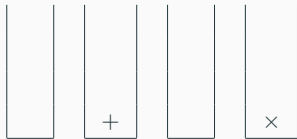
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 +`



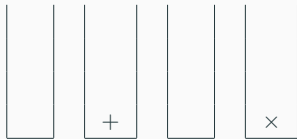
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7`



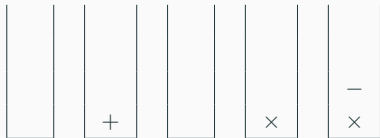
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérateur, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7`



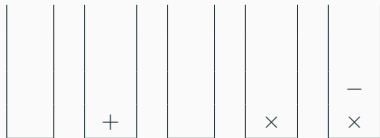
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7`



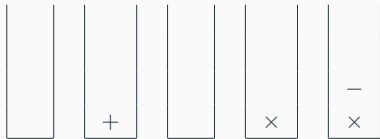
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2`



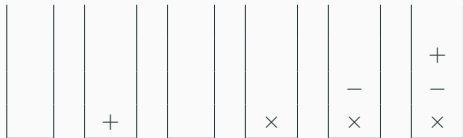
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérateur, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2`



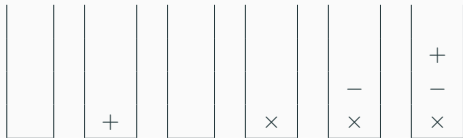
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2 2`



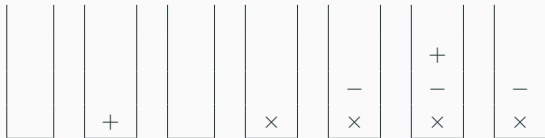
Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2 2 +`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2 2 + -`



Conversion infixe $\rightarrow$ postfixe : Exemple

Algorithme de conversion infixe $\rightarrow$ postfixe

On initialise `String result = ""`, puis on parcourt l'expression de gauche à droite :

- ▶ Si l'élément rencontré est un opérande, on l'ajoute au résultat
- ▶ Si c'est un opérateur, on l'empile
- ▶ Si c'est une "(", il n'y a rien à faire
- ▶ Si c'est une ")", on dépile un opérateur et on l'ajoute au résultat
- ▶ À la fin, la variable `result` contient la forme postfixe

Exemple : Forme infixe : $((3 + 2) \times (7 - (2 + 2)))$ `result = 3 2 + 7 2 2 + - ×`

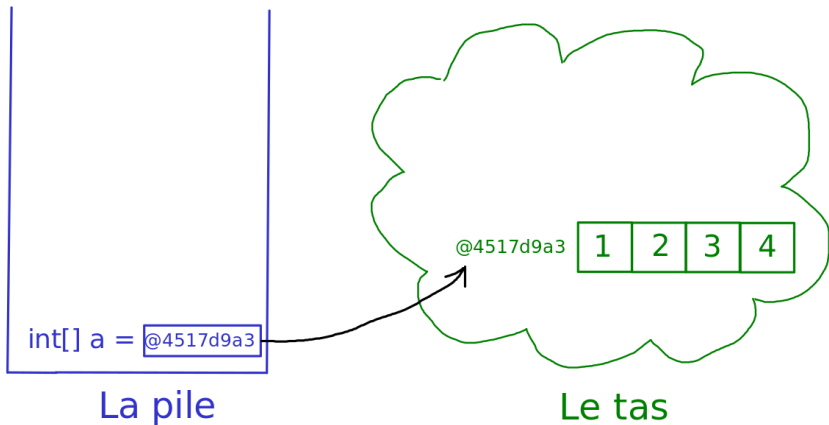


```
public static String convertToPostfix(String[] infixExpr) {  
    String result = "";  
    Stack<String> p = new Stack<String>();  
    for (String s : infixExpr) {  
        switch (s) {  
            case "(": // Parenthese ouvrante : ne rien faire  
                break;  
            case "+": case "-": case "*": case "/":  
                p.push(s);  
                break;  
            case ")":  
                result += p.pop();  
                break;  
            default: // Sinon : s est une operande  
                result += s;  
        }  
    }  
    return result;  
}
```

Retour sur la pile et le tas : tableaux de tableaux

Rappel : représentation mémoire d'un tableau

```
int[] a = {1, 2, 3, 4};
```



Rappel : représentation mémoire d'un tableau

```
int[] a = {1, 2, 3, 4};
```

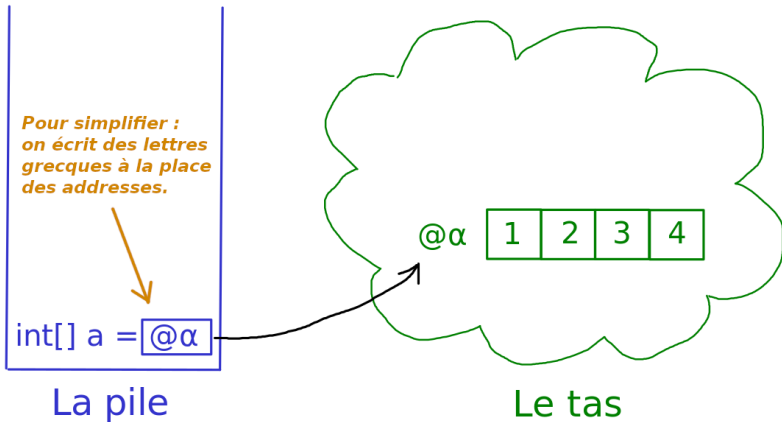


Tableau de tableaux — Exemple 1 : matrice

```
int[] [] tab = { {1, 2, 3},  
                  {4, 5, 6},  
                  {7, 8, 9} };
```

Tableau de tableaux — Exemple 1 : matrice

```
int[] [] tab = { {1, 2, 3},  
                 {4, 5, 6},  
                 {7, 8, 9} };
```

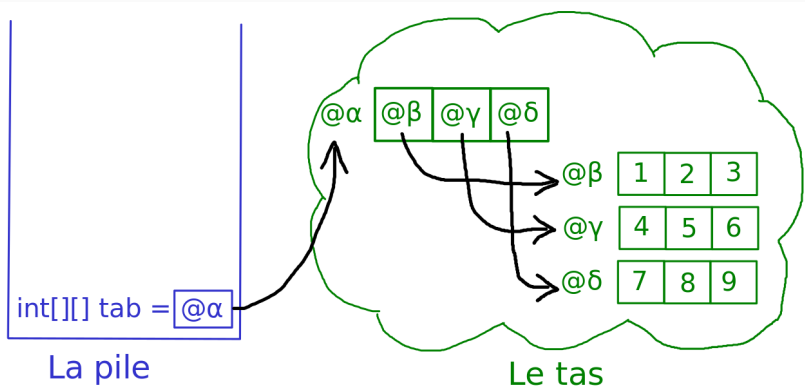


Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
int[] [] tab;  
tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    tab[i] = new int[i+1];
```

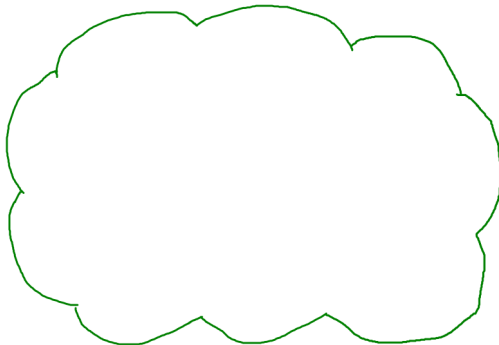
Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
► int[] [] tab;  
tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    tab[i] = new int[i+1];
```



int[][] tab = null

La pile



Le tas

Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
int[] [] tab;  
▶ tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    tab[i] = new int[i+1];
```

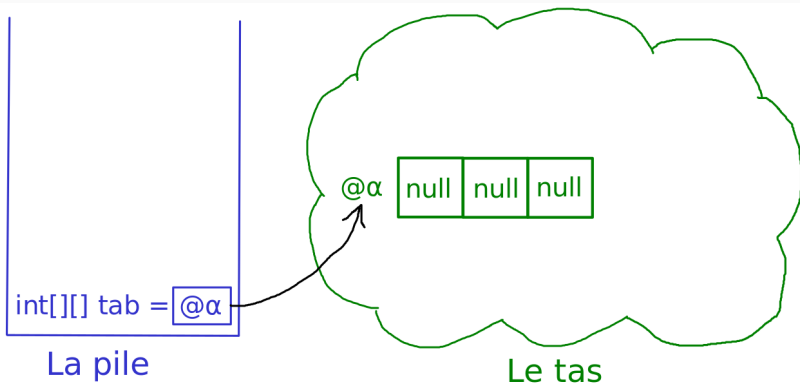


Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
int[] [] tab;  
tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    ► tab[i] = new int[i+1];
```

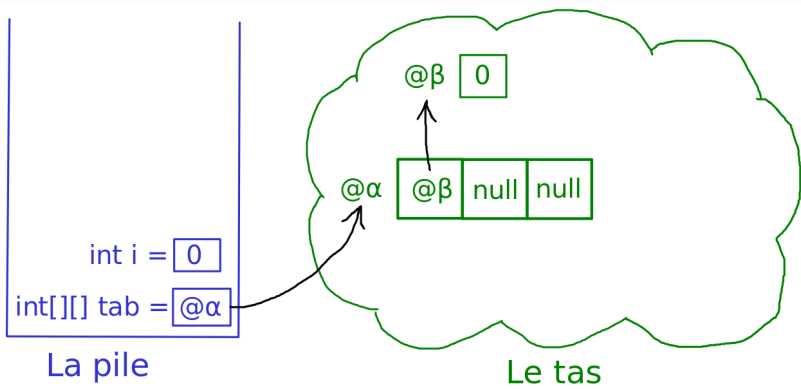


Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
int[] [] tab;  
tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    ► tab[i] = new int[i+1];
```

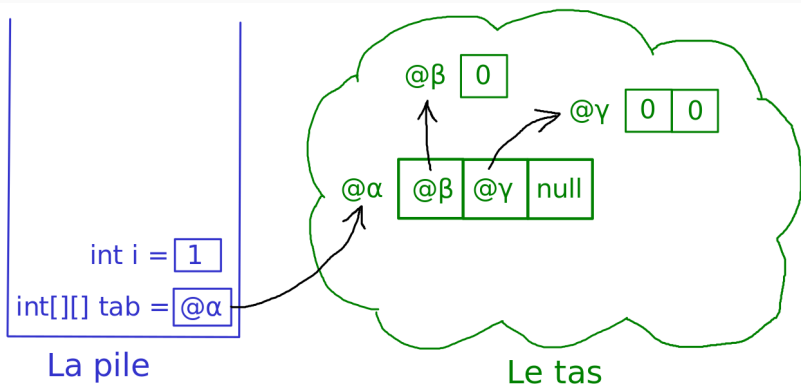


Tableau de tableaux — Exemple 2 : initialisation pas-à-pas

```
int[] [] tab;  
tab = new int[3] [];  
for (int i = 0; i < 3; i++)  
    ► tab[i] = new int[i+1];
```

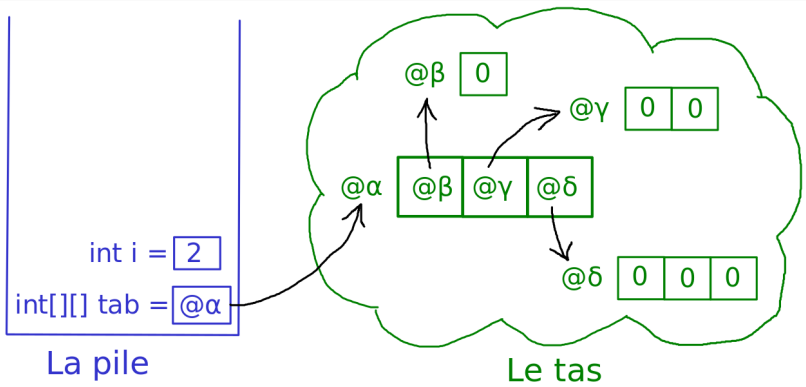


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    t[i] = u;
```

Tableau de tableaux — Exemple 3 : partage de référence

```
► int[] [] t = new int[4] [];  
  int[] u = {1, 1, 1};  
  for (int i = 0; i < 4; i++)  
    t[i] = u;
```

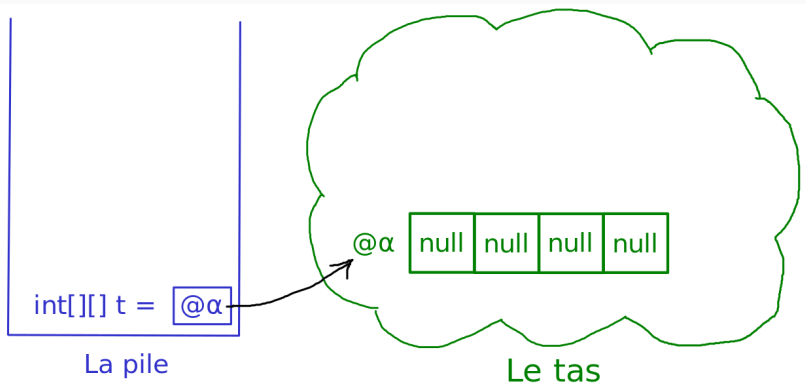


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
▶ int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    t[i] = u;
```

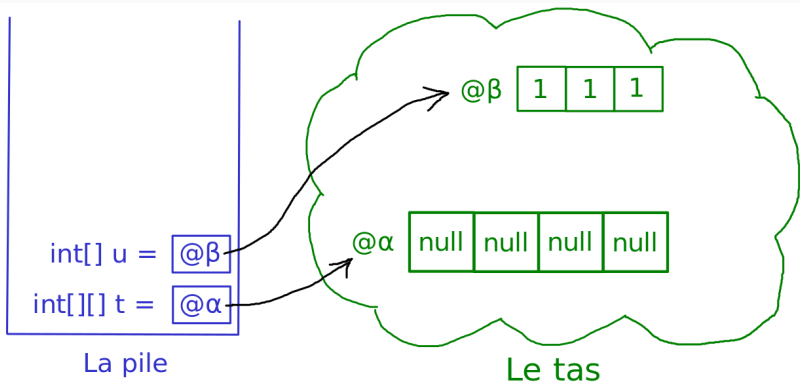


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    ► t[i] = u;
```

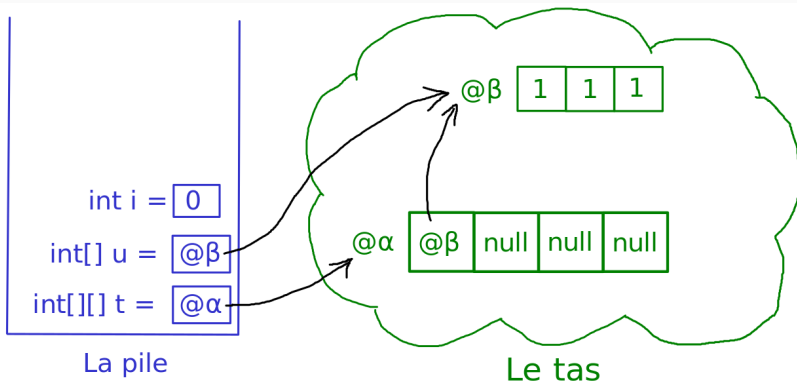


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    ► t[i] = u;
```

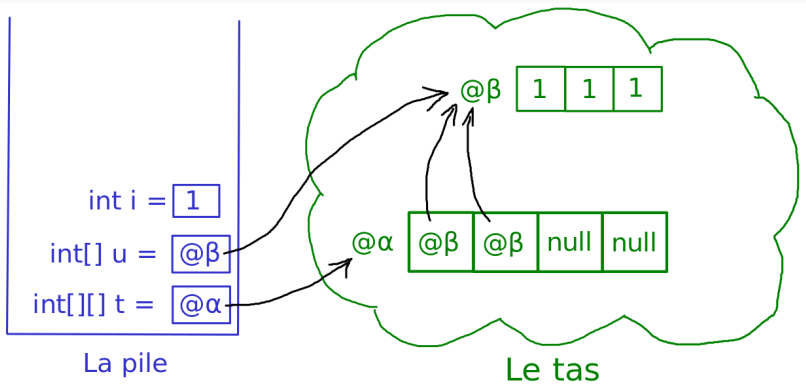


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    ► t[i] = u;
```

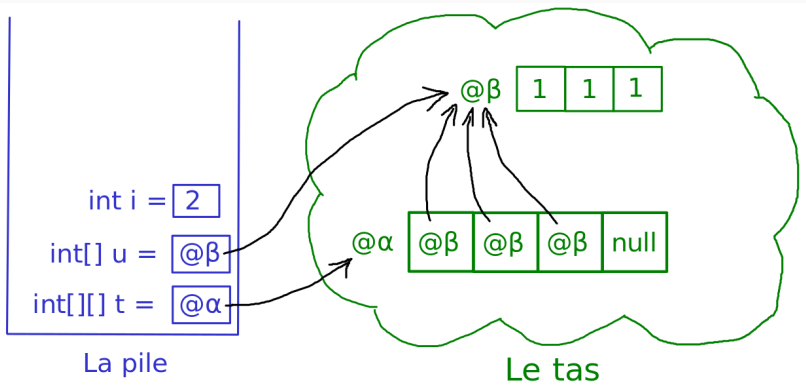
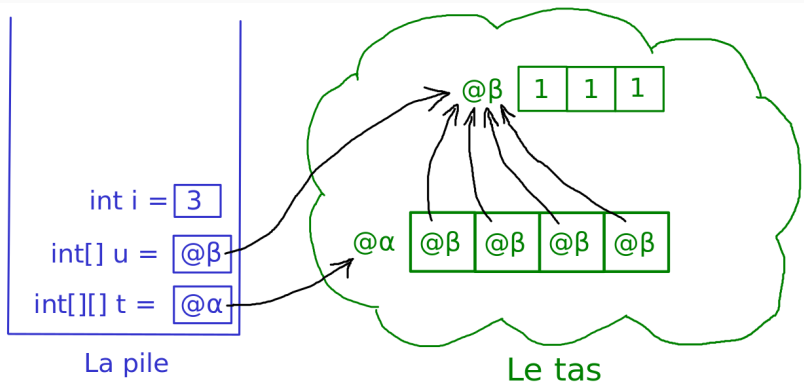


Tableau de tableaux — Exemple 3 : partage de référence

```
int[] [] t = new int[4] [];  
int[] u = {1, 1, 1};  
for (int i = 0; i < 4; i++)  
    ► t[i] = u;
```

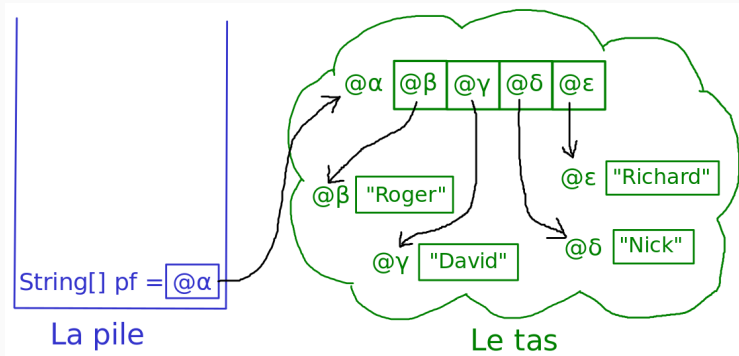


Exemple 4 : tableau de chaînes de caractères

```
String[] pf = { "Roger", "David", "Nick", "Richard" };
```

Exemple 4 : tableau de chaînes de caractères

```
String[] pf = { "Roger", "David", "Nick", "Richard" };
```



Pour davantage d'exemples

- ▶ Jetez un œil au fichier [TestTableaux.java](#) sur la page Moodle du cours.
Essayez de prévoir le comportement du programme avant de l'exécuter!
- ▶ Visualisez l'exécution pas-à-pas de vos propres programmes :

<http://pythontutor.com/java.html#mode=edit>

Attention : sur ce site, la pile est dessinée de haut en bas.

Un mot sur le ramasse-miettes (Garbage Collector – GC)

Parfois, certaines données présentes dans le tas deviennent **inaccessibles** :

```
int[] t = {1, 2, 3, 4};  
int[] u = {5, 6, 7, 8};  
t = u; // Ecrase l'ancienne valeur de t
```

Après l'exécution de ce code :

- ▶ Les deux variables `t` et `u` font référence au même tableau `{5, 6, 7, 8}`.
- ▶ Le tableau `{1, 2, 3, 4}` n'est plus référencé par aucune variable : on ne pourra plus jamais y accéder.

Un mot sur le ramasse-miettes (Garbage Collector – GC)

Parfois, certaines données présentes dans le tas deviennent **inaccessibles** :

```
int[] t = {1, 2, 3, 4};  
int[] u = {5, 6, 7, 8};  
t = u; // Ecrase l'ancienne valeur de t
```

Après l'exécution de ce code :

- ▶ Les deux variables `t` et `u` font référence au même tableau `{5, 6, 7, 8}`.
- ▶ Le tableau `{1, 2, 3, 4}` n'est plus référencé par aucune variable : on ne pourra plus jamais y accéder.

En Java

Un mécanisme appelé **Garbage Collector** (GC) libère automatiquement la mémoire inaccessible (ouf!)

En C/C++, il faudra libérer la mémoire à la main pour éviter les **fuites mémoires**...

Appels de fonction

Rappels sur les fonctions

Le rôle des fonctions

Une **fonction** (aussi appelée **méthode** dans le contexte orienté-objet) est un bloc de code réutilisable qui effectue une tâche spécifique. Elle peut éventuellement prendre des **paramètres** en entrée, et/ou **renvoyer** un résultat.

Rappels sur les fonctions

Le rôle des fonctions

Une **fonction** (aussi appelée **méthode** dans le contexte orienté-objet) est un bloc de code réutilisable qui effectue une tâche spécifique. Elle peut éventuellement prendre des **paramètres** en entrée, et/ou **renvoyer** un résultat.

En Java, une fonction est donnée par son **prototype**, qui indique le nom de la fonction, son type de retour, ainsi que le type et le nom de ses paramètres :

```
public static boolean isValid(String input, int maxLength)
```

Rappels sur les fonctions

Le rôle des fonctions

Une **fonction** (aussi appelée **méthode** dans le contexte orienté-objet) est un bloc de code réutilisable qui effectue une tâche spécifique. Elle peut éventuellement prendre des **paramètres** en entrée, et/ou **renvoyer** un résultat.

En Java, une fonction est donnée par son **prototype**, qui indique le nom de la fonction, son type de retour, ainsi que le type et le nom de ses paramètres :

```
public static boolean isValid(String input, int maxLength)
```

Les fonctions sont cruciales en programmation car elles permettent la **modularité** : le code est découpé en petit blocs qui réalisent chacun une tâche spécifique. Cela rend le code plus **lisible**, plus **facile à comprendre** et à **maintenir**.

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante :

Affichage :

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
▶ 12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 12

Affichage :

Debut main

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 13

Affichage :

Debut main

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
▶ 5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 5

Affichage :

Debut main

Debut f

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 6

Affichage :

Debut main

Debut f

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 2

Affichage :

Debut main

Debut f

Fonction g

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 7

Affichage :

Debut main

Debut f

Fonction g

Milieu f

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 8

Affichage :

Debut main

Debut f

Fonction g

Milieu f

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 2

Affichage :

Debut main

Debut f

Fonction g

Milieu f

Fonction g

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 9

Affichage :

Debut main

Debut f

Fonction g

Milieu f

Fonction g

Fin f

Appels de fonction : Exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```

Ligne courante : 14

Affichage :

Debut main

Debut f

Fonction g

Milieu f

Fonction g

Fin f

Fin main

Appels de fonction : le rôle de la pile

Qu'est-ce qui s'est passé ?

- ▶ Quand on appelle une fonction, on fait un saut vers la première ligne du code de cette fonction.

Appels de fonction : le rôle de la pile

Qu'est-ce qui s'est passé ?

- ▶ Quand on appelle une fonction, on fait un saut vers la première ligne du code de cette fonction.
- ▶ À la fin du code de la fonction appelée, l'exécution revient à l'endroit exact où elle avait été interrompue dans le code de la fonction appelante.

Appels de fonction : le rôle de la pile

Qu'est-ce qui s'est passé ?

- ▶ Quand on appelle une fonction, on fait un saut vers la première ligne du code de cette fonction.
- ▶ À la fin du code de la fonction appelée, l'exécution revient à l'endroit exact où elle avait été interrompue dans le code de la fonction appelante.
- ▶ Pour cela, il faut mémoriser le numéro de ligne courant de chaque fonction.

Appels de fonction : le rôle de la pile

Qu'est-ce qui s'est passé ?

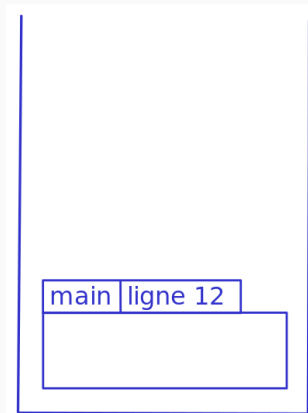
- ▶ Quand on **appelle** une fonction, on fait un **saut** vers la première ligne du code de cette fonction.
- ▶ À la fin du code de la fonction appelée, l'exécution revient **à l'endroit exact où elle avait été interrompue** dans le code de la fonction appelante.
- ▶ Pour cela, il faut mémoriser le **numéro de ligne courant** de chaque fonction.

Cette information va être stockée dans la pile.

C'est en fait la raison principale pour laquelle on utilise une structure de pile : on l'appelle parfois **pile d'appels** ou en anglais **call stack**.

Retour à notre exemple

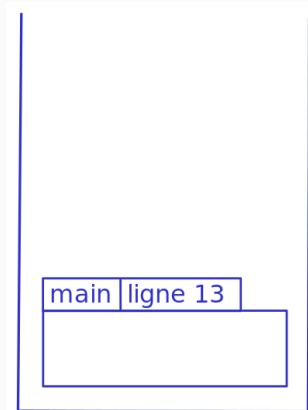
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

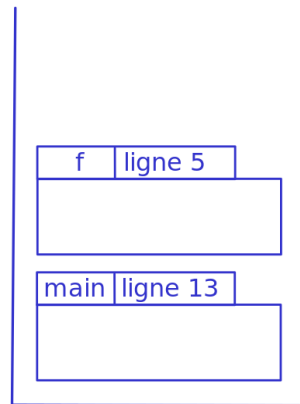
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

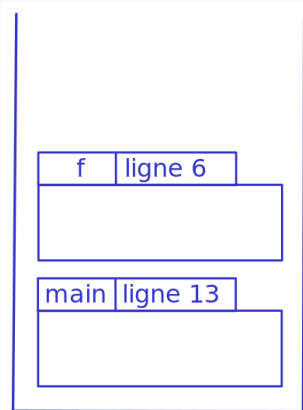
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

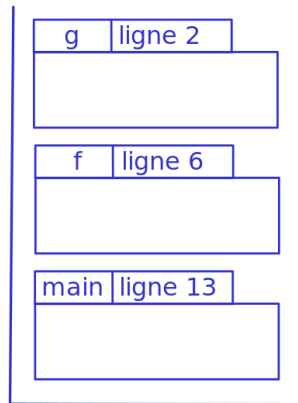
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

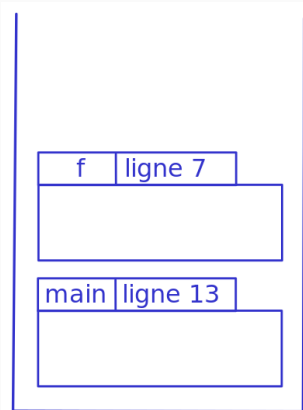
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

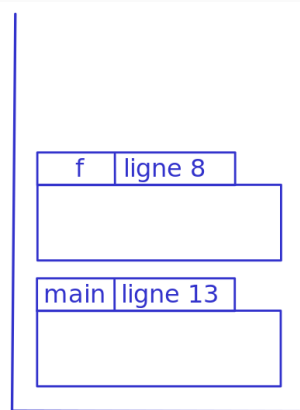
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

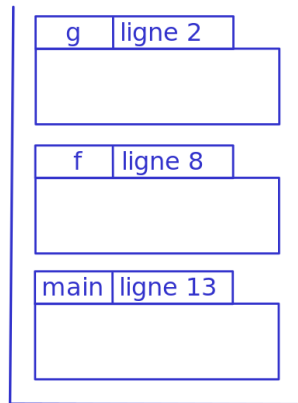
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

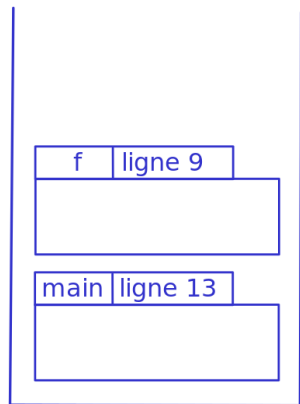
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

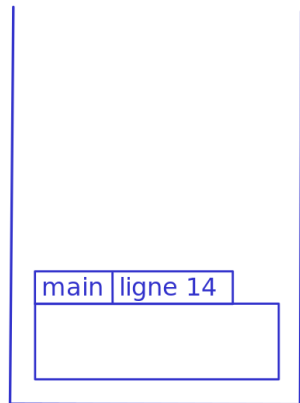
```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

Retour à notre exemple

```
1 public static void g() {  
2     System.out.println("Fonction g");  
3 }  
4 public static void f() {  
5     System.out.println("Debut f");  
6     g();  
7     System.out.println("Milieu f");  
8     g();  
9     System.out.println("Fin f");  
10 }  
11 public static void main(String[] args) {  
12     System.out.println("Debut main");  
13     f();  
14     System.out.println("Fin main");  
15 }
```



La pile

La pile d'appels dans les messages d'erreurs

En fait, vous avez déjà rencontré cette pile d'appels!

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestFonctions.h(TestFonctions.java:3)
    at TestFonctions.g(TestFonctions.java:7)
    at TestFonctions.f(TestFonctions.java:12)
    at TestFonctions.main(TestFonctions.java:20)
```

Ce message d'erreurs nous indique que :

- ▶ Il y a eu une erreur "ArithmeticException" à cause d'une division par zéro.
- ▶ Cette erreur s'est déclenchée dans la fonction `h`, à la ligne 3.
- ▶ La fonction `h` avait été appelée par la fonction `g`, à la ligne 7.
- ▶ La fonction `g` avait été appelée par la fonction `f`, à la ligne 12.
- ▶ La fonction `f` avait été appelée par la fonction `main`, à la ligne 20.

Fonctions : variables locales

Le **contexte d'exécution** d'une fonction est l'**environnement** dans lequel la fonction est exécutée. Il contient :

- ▶ Les **paramètres** (aussi appelés **arguments**) de la fonction.
- ▶ Les **variables locales** de la fonction.

Le **contexte d'exécution** d'une fonction est l'**environnement** dans lequel la fonction est exécutée. Il contient :

- ▶ Les **paramètres** (aussi appelés **arguments**) de la fonction.
- ▶ Les **variables locales** de la fonction.

Ces éléments ne sont accessibles qu'à l'intérieur de la fonction où ils sont définis; et ils n'existent que pendant l'exécution de la fonction.

Le **contexte d'exécution** d'une fonction est l'**environnement** dans lequel la fonction est exécutée. Il contient :

- ▶ Les **paramètres** (aussi appelés **arguments**) de la fonction.
- ▶ Les **variables locales** de la fonction.

Ces éléments ne sont accessibles qu'à l'intérieur de la fonction où ils sont définis; et ils n'existent que pendant l'exécution de la fonction.

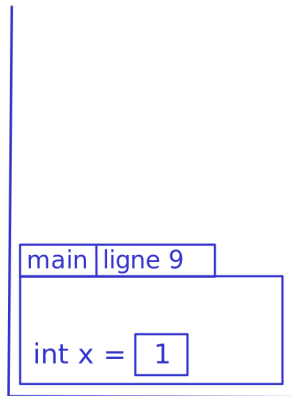
Ces données sont stockées sur la pile, dans le **bloc d'activation** (en anglais : **stack frame**) de la fonction.

Exemple : fonction avec des variables locales

```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```

Exemple : fonction avec des variables locales

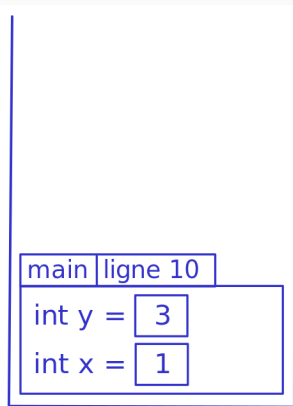
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

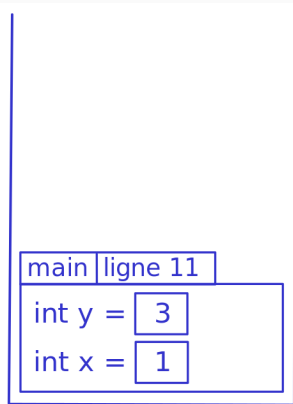
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```

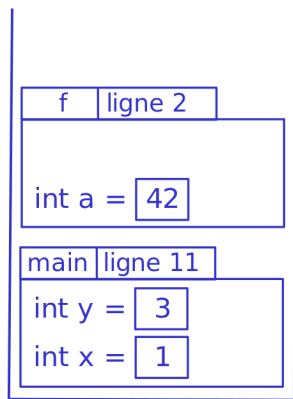


La pile

Exemple : fonction avec des variables locales



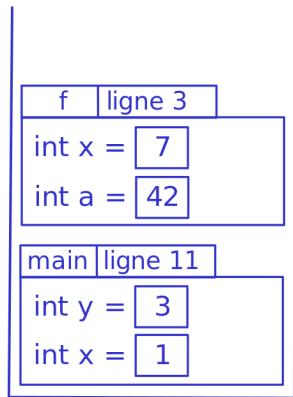
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

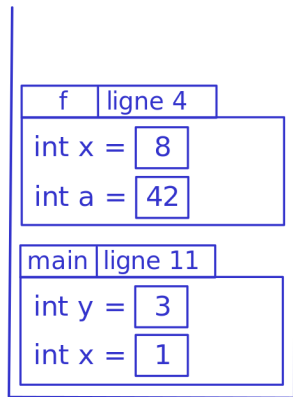
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

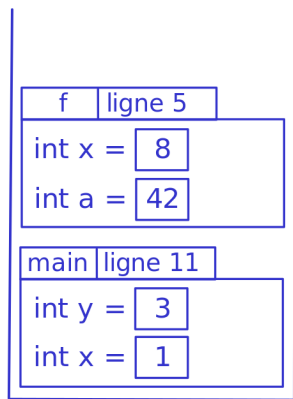
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

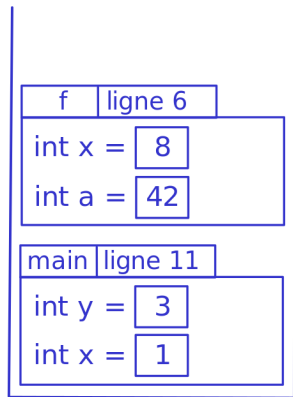
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

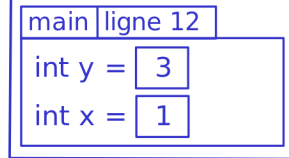
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

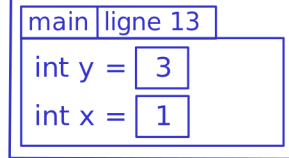
```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Exemple : fonction avec des variables locales

```
public static void f() {  
    int a = 42;  
    int x = 7;  
    x++;  
    System.out.print("Dans f : ");  
    System.out.println("x = " + x);  
}  
  
public static void main(String[] args) {  
    int x = 1;  
    int y = 3;  
    f();  
    System.out.print("Dans main : ");  
    System.out.println("x = " + x);  
}
```



La pile

Variables et erreurs de compilation

```
public static void f() {  
    System.out.println(x);  
}  
public static void main() {  
    int x = 23;  
    f();  
}
```

Le compilateur signale une erreur à la ligne 2 :

error: cannot find symbol

symbol: variable x

Variables et erreurs de compilation

```
public static void f() {  
    System.out.println(x);  
}  
public static void main() {  
    int x = 23;  
    f();  
}
```

Le compilateur signale une erreur à la ligne 2 :

```
error: cannot find symbol  
symbol: variable x
```

```
public static void g() {  
    int x = 4;  
    int x = 12;  
}
```

Le compilateur signale une erreur à la ligne 3 :

```
error: variable x is already  
defined
```

Ce qu'il nous reste à voir sur les fonctions

(RDV la semaine prochaine)

- ▶ Fonctions avec des paramètres
- ▶ Fonctions qui manipulent des tableaux / objets